Introduction to Robotics

Lecture 11: Dynamics in the Task Space and Force Control

Konstantinos Chatzilygeroudis - costashatz@upatras.gr

Department of Electrical and Computer Engineering
University of Patras

Template made by Panagiotis Papagiannopoulos

Dynamics of Open Chains

$$\underbrace{\boldsymbol{M}(\boldsymbol{q})}_{\text{“Mass Matrix”}} \ddot{\boldsymbol{q}} + \underbrace{\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})}_{\text{“Coriolis/Gravity Forces”}} = \underbrace{\boldsymbol{\tau}}_{\text{“Joint torques/forces”}} + \underbrace{\boldsymbol{F}_{\text{ext}}}_{\text{“External forces”}}$$

- $\boldsymbol{q}$ are the joint angles
- we will ignore $\boldsymbol{F}_{\text{ext}}$ for now
- $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \underbrace{\boldsymbol{c}(\boldsymbol{q}, \dot{\boldsymbol{q}})}_{\text{“Coriolis forces”}} + \underbrace{\boldsymbol{g}(\boldsymbol{q})}_{\text{“Gravity forces”}}$
- This equation is also called the *“Manipulator Equation”*

Forward and Inverse Dynamics of Open Chains

- **Inverse Dynamics:** Given a desired acceleration $\ddot{\boldsymbol{q}}_d$, we can compute the wrench to achieve it by

$$\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}}_d + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) - \boldsymbol{F}_{\text{ext}}$$

  - We can use this for control or trajectory synthesis
  - We answer the question: *"What are the joint torques/forces required to exert in order to achieve some desired joint accelerations?"*
- **Forward Dynamics:** Given some realized joint torques/forces $\boldsymbol{\tau}$, we can compute the joint accelerations generated by them

$$\ddot{\boldsymbol{q}} = \boldsymbol{M}^{-1}(\boldsymbol{q})\Big(\boldsymbol{\tau} + \boldsymbol{F}_{\text{ext}} - \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\Big)$$

  - We can use this for forward simulation or estimation
  - We answer the question: *"What are the joint accelerations that are generated by the application of some specific joint torques/forces?"*

Newton-Euler Inverse Dynamics Algorithm

Base frame quantities:

$$\mathcal{V}_0 = \mathbf{0}$$
$$\dot{\mathcal{V}}_0 = \begin{bmatrix} \mathbf{0} & -\boldsymbol{g} \end{bmatrix}^T$$

where n the DoFs of the robot, and $\boldsymbol{g} \in \mathbb{R}^3$ is the gravity vector.

Forward Pass $(i = 1 \rightarrow n)$

$$\boldsymbol{T}_{i,i-1} = \exp(-\mathcal{A}_i \boldsymbol{q}_i)\boldsymbol{M}_{i,i-1}$$
$$\mathcal{V}_i = \mathcal{A}_i \dot{\boldsymbol{q}}_i + [Ad_{\boldsymbol{T}_{i,i-1}}]\mathcal{V}_{i-1}$$
$$\dot{\mathcal{V}}_i = \mathcal{A}_i \ddot{\boldsymbol{q}}_i + [Ad_{\boldsymbol{T}_{i,i-1}}]\dot{\mathcal{V}}_{i-1} + [ad_{\mathcal{V}_i}]\mathcal{A}_i \dot{\boldsymbol{q}}_i$$

Backward Pass $(i = n \rightarrow 1)$

$$\mathcal{F}_i = [Ad_{\boldsymbol{T}_{i+1,i}}]^T \mathcal{F}_{i+1} + \mathcal{G}_i \dot{\mathcal{V}}_i - [ad_{\mathcal{V}_i}]^T \mathcal{G}_i \mathcal{V}_i$$
$$\boldsymbol{\tau}_i = \mathcal{F}_i^T \mathcal{A}_i$$

where $\mathcal{F}_{n+1}$ is the wrench the end-effector applies to the environment.

SRBD in different frames - Reminder

We can express the dynamics in other frames than the center of mass of frame $\{b\}$. Let's call another frame $\{a\}$. The main "conversion" that we need is to transform the spatial inertia matrix from one frame to the other. We can do the same trick with the kinetic energy:

$$\begin{aligned}\frac{1}{2}\mathcal{V}_a^T\mathcal{G}_a\mathcal{V}_a &= \frac{1}{2}\mathcal{V}_b^T\mathcal{G}_b\mathcal{V}_b \\ &= \frac{1}{2}([Ad_{\boldsymbol{T}_{ba}}]\mathcal{V}_a)^T\mathcal{G}_b[Ad_{\boldsymbol{T}_{ba}}]\mathcal{V}_a \\ &= \frac{1}{2}\mathcal{V}_a^T[Ad_{\boldsymbol{T}_{ba}}]^T\mathcal{G}_b[Ad_{\boldsymbol{T}_{ba}}]\mathcal{V}_a\end{aligned}$$

Thus, we have:

$$\begin{aligned}\mathcal{G}_a &= [Ad_{\boldsymbol{T}_{ba}}]^T\mathcal{G}_b[Ad_{\boldsymbol{T}_{ba}}] \\ \mathcal{F}_a &= \mathcal{G}_a\dot{\mathcal{V}}_a - [ad_{\mathcal{V}_a}]^T\mathcal{G}_a\mathcal{V}_a\end{aligned}$$

Generic Newton-Euler Inverse Dynamics

What if the body frames do not coincide with the CoM frames?

Generic Newton-Euler Inverse Dynamics

What if the body frames do not coincide with the CoM frames?

- Now each frame $\{i\}$ is not located at the CoM of the body i
- $\boldsymbol{M}_{i,j}$ are computed based on the "new" body frames
- Similarly for $\mathcal{A}_i$'s

Generic Newton-Euler Inverse Dynamics

What if the body frames do not coincide with the CoM frames?

- Now each frame $\{i\}$ is not located at the CoM of the body i
- $\boldsymbol{M}_{i,j}$ are computed based on the "new" body frames
- Similarly for $\mathcal{A}_i$'s
- We compute the spatial inertia matrix in the body frame

Generic Newton-Euler Inverse Dynamics

What if the body frames do not coincide with the CoM frames?

- Now each frame $\{i\}$ is not located at the CoM of the body i
- $\boldsymbol{M}_{i,j}$ are computed based on the "new" body frames
- Similarly for $\mathcal{A}_i$'s
- We compute the spatial inertia matrix in the body frame
- We know how: $\mathcal{G}_i = [Ad_{\boldsymbol{T}_{bi}}]^T \mathcal{G}_b [Ad_{\boldsymbol{T}_{bi}}]$

Generic Newton-Euler Inverse Dynamics

What if the body frames do not coincide with the CoM frames?

- Now each frame $\{i\}$ is not located at the CoM of the body i
- $\boldsymbol{M}_{i,j}$ are computed based on the "new" body frames
- Similarly for $\mathcal{A}_i$'s
- We compute the spatial inertia matrix in the body frame
- We know how: $\mathcal{G}_i = [Ad_{\boldsymbol{T}_{bi}}]^T \mathcal{G}_b [Ad_{\boldsymbol{T}_{bi}}]$
- We proceed as usual!

Closed Form Derivation

Now we can compute:

$$
\begin{aligned}
\mathcal{V}_{\text{all}} &= \mathcal{L}(\boldsymbol{q})\Big(\mathcal{A}_{\text{all}}\dot{\boldsymbol{q}} + \mathcal{V}_{\text{base}}\Big) \\
\dot{\mathcal{V}}_{\text{all}} &= \mathcal{L}(\boldsymbol{q})\Big(\mathcal{A}_{\text{all}}\ddot{\boldsymbol{q}} - [ad_{\mathcal{A}_{\text{all}}\dot{\boldsymbol{q}}}]\mathcal{W}(\boldsymbol{q})\mathcal{V} - [ad_{\mathcal{A}_{\text{all}}\dot{\boldsymbol{q}}}]\mathcal{V}_{\text{base}} + \dot{\mathcal{V}}_{\text{base}}\Big) \\
\mathcal{F}_{\text{all}} &= \mathcal{L}^T(\boldsymbol{q})\Big(\mathcal{G}_{\text{all}}\dot{\mathcal{V}}_{\text{all}} - [ad_{\mathcal{V}_{\text{all}}}]^T\mathcal{G}_{\text{all}}\mathcal{V}_{\text{all}} + \mathcal{F}_{\text{tip}}\Big) \\
\boldsymbol{\tau} &= \mathcal{A}_{\text{all}}^T\mathcal{F}_{\text{all}}
\end{aligned}
$$

And:

$$
\begin{aligned}
\boldsymbol{M}(\boldsymbol{q}) &= \mathcal{A}_{\text{all}}^T\mathcal{L}^T(\boldsymbol{q})\mathcal{G}_{\text{all}}\mathcal{L}(\boldsymbol{q})\mathcal{A}_{\text{all}} \\
\boldsymbol{c}(\boldsymbol{q},\dot{\boldsymbol{q}}) &= -\mathcal{A}_{\text{all}}^T\mathcal{L}^T(\boldsymbol{q})\Big(\mathcal{G}_{\text{all}}\mathcal{L}(\boldsymbol{q})[ad_{\mathcal{A}_{\text{all}}\dot{\boldsymbol{q}}}]\mathcal{W}(\boldsymbol{q}) + [ad_{\mathcal{V}_{\text{all}}}]^T\mathcal{G}_{\text{all}}\Big)\mathcal{L}(\boldsymbol{q})\mathcal{A}_{\text{all}}\dot{\boldsymbol{q}} \\
\boldsymbol{g}(\boldsymbol{q}) &= \mathcal{A}_{\text{all}}^T\mathcal{L}^T(\boldsymbol{q})\mathcal{G}_{\text{all}}\mathcal{L}(\boldsymbol{q})\dot{\mathcal{V}}_{\text{base}}
\end{aligned}
$$

Dynamics of Open Chains: Verdict

- The Lagrangian formulation is suitable only for simple systems
- The Newton-Euler formulation provides an efficient recursive algorithm
- The "closed-form" formulation is still a recursive algorithm
- There exist more sophisticated and more efficient algorithms (e.g. Roy Featherstone's Rigid Body Dynamics Algorithms book)
- We can even analytically compute the gradients of the dynamics equations

What about torque/force control?

How can we control with torque actuators?

What about torque/force control?

How can we control with torque actuators?

- **Feedforward/Open-loop control:**

$$\boldsymbol{\tau}(t) = \boldsymbol{M}(\boldsymbol{q}_d(t))\ddot{\boldsymbol{q}}_d(t) + \boldsymbol{C}(\boldsymbol{q}_d(t), \dot{\boldsymbol{q}}_d(t))$$

- **Feedback control (PID):**

$$\boldsymbol{\tau}(t) = K_p\boldsymbol{q}_e(t) + K_i\int_0^t \boldsymbol{q}_e(t)dt + K_d\dot{\boldsymbol{q}}_e(t), \quad K_p, K_i, K_d > 0$$

- **Feedforward plus feedback linearizing controller (Inverse Dynamics Controller):**

$$\begin{aligned}\boldsymbol{\tau}(t) =& \boldsymbol{M}(\boldsymbol{q}(t))\Big(\ddot{\boldsymbol{q}}_d(t) + K_p\boldsymbol{q}_e(t) + K_i\int_0^t \boldsymbol{q}_e(t)dt + K_d\dot{\boldsymbol{q}}_e(t)\Big)\\ &+ \boldsymbol{C}(\boldsymbol{q}_d(t), \dot{\boldsymbol{q}}_d(t)), \quad K_p, K_i, K_d > 0\end{aligned}$$

- **Gravity Compensation controller:**

$$\boldsymbol{\tau}(t) = K_p\boldsymbol{q}_e(t) + K_i\int_0^t \boldsymbol{q}_e(t)dt + K_d\dot{\boldsymbol{q}}_e(t) + \boldsymbol{g}(\boldsymbol{q}(t)), \quad K_p, K_i, K_d > 0$$

Task-Space Robot Dynamics

How can we define the manipulator equation in task-space?

Task-Space Robot Dynamics

How can we define the manipulator equation in task-space?

- We have: $\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{\tau}$
- And: $\mathcal{V} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$
- Let's differentiate it:

Task-Space Robot Dynamics

How can we define the manipulator equation in task-space?

- We have: $\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{\tau}$
- And: $\mathcal{V} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$
- Let's differentiate it: $\dot{\mathcal{V}} = \dot{\boldsymbol{J}}\dot{\boldsymbol{q}} + \boldsymbol{J}\ddot{\boldsymbol{q}}$
- Let's solve for $\dot{\boldsymbol{q}}$: $\dot{\boldsymbol{q}} = \boldsymbol{J}^{-1}\mathcal{V}$

Task-Space Robot Dynamics

How can we define the manipulator equation in task-space?

- We have: $\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{\tau}$
- And: $\mathcal{V} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$
- Let's differentiate it: $\dot{\mathcal{V}} = \dot{\boldsymbol{J}}\dot{\boldsymbol{q}} + \boldsymbol{J}\ddot{\boldsymbol{q}}$
- Let's solve for $\dot{\boldsymbol{q}}$: $\dot{\boldsymbol{q}} = \boldsymbol{J}^{-1}\mathcal{V}$
- Let's solve for $\ddot{\boldsymbol{q}}$: $\ddot{\boldsymbol{q}} = \boldsymbol{J}^{-1}\dot{\mathcal{V}} - \boldsymbol{J}^{-1}\dot{\boldsymbol{J}}\boldsymbol{J}^{-1}\mathcal{V}$

Task-Space Robot Dynamics

How can we define the manipulator equation in task-space?

- We have: $\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \boldsymbol{\tau}$
- And: $\mathcal{V} = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$
- Let's differentiate it: $\dot{\mathcal{V}} = \dot{\boldsymbol{J}}\dot{\boldsymbol{q}} + \boldsymbol{J}\ddot{\boldsymbol{q}}$
- Let's solve for $\dot{\boldsymbol{q}}$: $\dot{\boldsymbol{q}} = \boldsymbol{J}^{-1}\mathcal{V}$
- Let's solve for $\ddot{\boldsymbol{q}}$: $\ddot{\boldsymbol{q}} = \boldsymbol{J}^{-1}\dot{\mathcal{V}} - \boldsymbol{J}^{-1}\dot{\boldsymbol{J}}\boldsymbol{J}^{-1}\mathcal{V}$
- Now let's put everything together:

$$\begin{aligned}
\boldsymbol{\tau} &= \boldsymbol{M}\big(\boldsymbol{J}^{-1}\dot{\mathcal{V}} - \boldsymbol{J}^{-1}\dot{\boldsymbol{J}}\boldsymbol{J}^{-1}\mathcal{V}\big) + \boldsymbol{C}(\boldsymbol{q}, \boldsymbol{J}^{-1}\mathcal{V}) \\
\boldsymbol{J}^{-T}\boldsymbol{\tau} &= \boldsymbol{J}^{-T}\boldsymbol{M}\big(\boldsymbol{J}^{-1}\dot{\mathcal{V}} - \boldsymbol{J}^{-1}\dot{\boldsymbol{J}}\boldsymbol{J}^{-1}\mathcal{V}\big) + \boldsymbol{J}^{-T}\boldsymbol{C}(\boldsymbol{q}, \boldsymbol{J}^{-1}\mathcal{V}) \\
\boldsymbol{J}^{-T}\boldsymbol{\tau} &= \boldsymbol{J}^{-T}\boldsymbol{M}\boldsymbol{J}^{-1}\dot{\mathcal{V}} - \boldsymbol{J}^{-T}\boldsymbol{M}\boldsymbol{J}^{-1}\dot{\boldsymbol{J}}\boldsymbol{J}^{-1}\mathcal{V} + \boldsymbol{J}^{-T}\boldsymbol{C}(\boldsymbol{q}, \boldsymbol{J}^{-1}\mathcal{V})
\end{aligned}$$

Task-Space Robot Dynamics (2)

Manipulator Equation in Task-Space:

$$\underbrace{\boldsymbol{\Lambda}(\boldsymbol{q})}_{\text{“Task-Space Inertia Matrix”}} \underbrace{\dot{\mathcal{V}}}_{\text{“End-effector acceleration”}} + \underbrace{\boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V})}_{\text{“Coriolis/Gravity Forces”}} = \underbrace{\mathcal{F}}_{\text{“Wrench at end-effector”}}$$

where:

$$\begin{aligned}
\boldsymbol{\Lambda}(\boldsymbol{q}) &= \boldsymbol{J}(\boldsymbol{q})^{-T} \boldsymbol{M}(\boldsymbol{q}) \boldsymbol{J}(\boldsymbol{q})^{-1} \\
\boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V}) &= \boldsymbol{J}(\boldsymbol{q})^{-T} \boldsymbol{C}(\boldsymbol{q}, \boldsymbol{J}(\boldsymbol{q})^{-1}\mathcal{V}) - \boldsymbol{\Lambda}(\boldsymbol{q}) \dot{\boldsymbol{J}}(\boldsymbol{q}) \boldsymbol{J}(\boldsymbol{q})^{-1} \mathcal{V} \\
\mathcal{F} &= \boldsymbol{J}^{-T} \boldsymbol{\tau}
\end{aligned}$$

Task-Space Robot Dynamics (2)

Manipulator Equation in Task-Space:

$$\underbrace{\boldsymbol{\Lambda}(\boldsymbol{q})}_{\text{“Task-Space Inertia Matrix”}} \underbrace{\dot{\mathcal{V}}}_{\text{“End-effector acceleration”}} + \underbrace{\boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V})}_{\text{“Coriolis/Gravity Forces”}} = \underbrace{\mathcal{F}}_{\text{“Wrench at end-effector”}}$$

where:

$$\begin{aligned}
\boldsymbol{\Lambda}(\boldsymbol{q}) &= \boldsymbol{J}(\boldsymbol{q})^{-T} \boldsymbol{M}(\boldsymbol{q}) \boldsymbol{J}(\boldsymbol{q})^{-1} \\
\boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V}) &= \boldsymbol{J}(\boldsymbol{q})^{-T} \boldsymbol{C}(\boldsymbol{q}, \boldsymbol{J}(\boldsymbol{q})^{-1}\mathcal{V}) - \boldsymbol{\Lambda}(\boldsymbol{q}) \dot{\boldsymbol{J}}(\boldsymbol{q}) \boldsymbol{J}(\boldsymbol{q})^{-1} \mathcal{V} \\
\mathcal{F} &= \boldsymbol{J}^{-T} \boldsymbol{\tau}
\end{aligned}$$

Problems?

Task-Space Robot Dynamics (2)

Manipulator Equation in Task-Space:

$$\underbrace{\boldsymbol{\Lambda}(\boldsymbol{q})}_{\text{“Task-Space Inertia Matrix”}} \underbrace{\dot{\mathcal{V}}}_{\text{“End-effector acceleration”}} + \underbrace{\boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V})}_{\text{“Coriolis/Gravity Forces”}} = \underbrace{\mathcal{F}}_{\text{“Wrench at end-effector”}}$$

where:

$$\begin{aligned}
\boldsymbol{\Lambda}(\boldsymbol{q}) &= \boldsymbol{J}(\boldsymbol{q})^{-T}\boldsymbol{M}(\boldsymbol{q})\boldsymbol{J}(\boldsymbol{q})^{-1} \\
\boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V}) &= \boldsymbol{J}(\boldsymbol{q})^{-T}\boldsymbol{C}(\boldsymbol{q}, \boldsymbol{J}(\boldsymbol{q})^{-1}\mathcal{V}) - \boldsymbol{\Lambda}(\boldsymbol{q})\dot{\boldsymbol{J}}(\boldsymbol{q})\boldsymbol{J}(\boldsymbol{q})^{-1}\mathcal{V} \\
\mathcal{F} &= \boldsymbol{J}^{-T}\boldsymbol{\tau}
\end{aligned}$$

Problems? We need the Jacobian to be invertible! Jacobian pseudoinverse does not work that well in practice!

Force Control in Task-Space

Two Options:

- Any feedback controller in task-space quantities (aka, $SE(3)$), and use the task-space dynamics analogously to the joint space dynamics. For example:

$$\boldsymbol{\tau}(t) = \boldsymbol{J}_b(\boldsymbol{q})^T \Big(\boldsymbol{\Lambda}(\boldsymbol{q}) \big(K_p \mathcal{X}_e(t) + K_i \int_0^t \mathcal{X}_e(t) dt \big) + \boldsymbol{\eta}(\boldsymbol{q}, \mathcal{V}) \Big),$$

$$K_p, K_i > 0$$

- Any feedback controller in task-space quantities (aka, $SE(3)$), and use no dynamics or joint-space dynamics. For example:

$$\boldsymbol{\tau} = \boldsymbol{J}_w^T \mathcal{F}_d + \boldsymbol{M}(\boldsymbol{q}_d(t)) \ddot{\boldsymbol{q}}_d(t) + \boldsymbol{C}(\boldsymbol{q}_d(t), \dot{\boldsymbol{q}}_d(t))$$

where $\mathcal{F}_d$ is computed with a task space P(ID) controller in world frame, and $\boldsymbol{q}_d(t)$ is one joint trajectory that corresponds to the desired end-effector trajectory.

Null-Space Controllers

When controlling redundant robots in task-space:

- there are multiple solutions at each time-step
- all controllers "optimize" for 1-step in the future
- we might end-up in bad situations!

Null-space controllers to the rescue:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{control}} + \underbrace{(\boldsymbol{I} - \boldsymbol{J}^T(\boldsymbol{q})\boldsymbol{J}^{T\dagger}(\boldsymbol{q}))\boldsymbol{\tau}_{\text{reg}}}_{\boldsymbol{\tau}_{\text{null}}}$$

where:

- $\boldsymbol{J}^{T\dagger}$ is the pseudoinverse of $\boldsymbol{J}^T$
- $\boldsymbol{\tau}_{\text{reg}}$ is a regularizing control input
- $\boldsymbol{\tau}_{\text{null}}$ does not affect completion of $\boldsymbol{\tau}_{\text{control}}$
- works for velocity control as well!

Bibliography

Chapter 8 and 11 from **Modern Robotics: Mechanics, Planning, and Control**, *Kevin M. Lynch and Frank C. Park*, 2017, Cambridge University Press. ebook

Thank you

- **Any Questions?**
- **Office Hours**:
  - **Tue-Thu** *(10:00-12:00)*
  - 24/7 by email (`costashatz@upatras.gr`, subject: *ECE_IR_AM*)
- **Material and Announcements**

Laboratory of Automation & Robotics